Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün.

Laboratoriya işi № 2

Tələbənin adı və soyadı: Ülvi Əliyev

Qrup: KOM25A

Müəllim: Şamil Hümbətov

Tarix: 20 oktyabr 2026

2026

Məqsəd

Bu laboratoriya işinin məqsədi tələbələrin Linked List, Stack və Queue verilənlər strukturlarının nəzəri əsaslarını öyrənməsi, onları sıfırdan C++ dilində (STL istifadə etmədən) implementasiya etməsi, eksperimental sınaqlar aparması və nəticələri elmi əsaslarla təhlil etməsidir.

Öyrənmə Nəticələri

Laboratoriya işini uğurla tamamlayan tələbə:

• Linked List, Stack və Queue strukturlarının iş prinsiplərini izah edə bilir;

• Verilənlər strukturlarını STL konteynerlərindən istifadə etmədən implementasiya edə bilir;

• Real həyat problemlərini uyğun verilənlər strukturları vasitəsilə modelləşdirə bilir;

• Eksperimental nəticələri cədvəl və qrafiklər vasitəsilə təqdim edə bilir;

• Teorik və praktiki mürəkkəblikləri müqayisə edə bilir;

• Performansa təsir edən sistem parametrlərini müəyyən edə bilir.

1. Giriş

Verilənlər strukturları proqram təminatının səmərəli və sürətli işləməsi üçün əsas bünövrə təşkil edir. Bu laboratoriya işində xətti verilənlər strukturlarından olan Linked List (Təkbağlı Siyahı), Stack (Yığın) və Queue (Növbə) mexanizmləri araşdırılmışdır. Məqsəd C++ standart kitabxanasının hazır konteynerlərindən (std::list, std::stack, std::queue) istifadə etmədən, dinamik yaddaş idarəetməsi və göstəricilər (pointers) vasitəsilə bu strukturları sıfırdan qurmaq və onların icra vaxtlarını praktiki olaraq ölçməkdir.

2. Tapşırıq 1: Smart Music Playlist (Linked List)

A. Araşdırma

1. Array və Linked List arasında əsas fərqlər: Array-də elementlər yaddaşda ardıcıl (contiguous) bloklarda saxlanılır. Linked List-də isə elementlər yaddaşın müxtəlif yerlərində dağınıq yerləşir və pointer-lər vasitəsilə bağlanır. Array-in ölçüsü sabitlənir, Linked List-in ölçüsü isə tamamilə dinamikdir. Array-də elementə ani müraciət O(1) olduğu halda, Linked List-də O(N) təşkil edir.

2. Musiqi playlistləri üçün Linked List-in uyğunluğu: Musiqi playlistləri sabit ölçüyə malik deyil. İstifadəçi gün ərzində siyahıya yeni mahnılar əlavə edir və ya silir. Linked List yaddaşda əvvəlcədən böyük yer ayırmağı tələb etmədiyi üçün resurslardan səmərəli istifadə edir və əlavə etmə/silmə əməliyyatlarını sürətli yerinə yetirir.

3. Singly və Doubly Linked List müqayisəsi: Singly Linked List daha az yaddaş tələb edir, lakin yalnız bir istiqamətdə hərəkət etmək mümkündür. Doubly Linked List-də həm növbəti (next), həm də əvvəlki (prev) göstərici saxlandığı üçün playlistdə 'Next' və 'Previous' keçidləri rahatlaşır, lakin əlavə yaddaş sərfi yaradır.

4. Linked List-də axtarışın yavaş olma səbəbi: Array elementləri yaddaşda yan-yana düzüldüyü üçün indeksi bilməklə birbaşa müraciət olunur və CPU cache-dən effektiv istifadə edilir. Linked List-də düyünlər fərqli ünvanlarda yerləşdiyindən axtarış zamanı 1-ci düyündən başlayaraq göstəriciləri ardıcıl izləmək O(N) tələb edir.

5. Playlistdə sıra dəyişdirilməsi üçün əlverişli struktur: Mahnıların sırasını dəyişmək üçün Doubly Linked List daha əlverişlidir. Array-də sıra dəyişdikdə bütün elementləri sürüşdürmək O(N) tələb olunur, Doubly Linked List-də isə sadəcə yerini dəyişən düyünün və qonşularının pointer-lərini yenidən bağlamaq kifayətdir.

B. Proqramlaşdırma (C++ Kod Nümunəsi)

```
struct SongNode {
    int songID;
    std::string songName;
    std::string artist;
    int duration;
    SongNode* next;

    SongNode(int id, std::string name, std::string art, int dur)
        : songID(id), songName(name), artist(art), duration(dur), next(nullptr) {}
};

class Playlist {
private:
    SongNode* head;
    int count;

public:
    Playlist() : head(nullptr), count(0) {}

    void addSong(int id, std::string name, std::string art, int dur) {
        SongNode* newNode = new SongNode(id, name, art, dur);
        if (!head) {
            head = newNode;
        } else {
            SongNode* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
            temp->next = newNode;
        }
        count++;
    }

    void deleteSong(int id) {
        if (!head) return;
        if (head->songID == id) {
            SongNode* temp = head;
            head = head->next;
```

```
            delete temp;
            count--;
            return;
        }
        SongNode* current = head;
        while (current->next && current->next->songID != id) {
            current = current->next;
        }
        if (current->next) {
            SongNode* temp = current->next;
            current->next = current->next->next;
            delete temp;
            count--;
        }
    }

    void searchSong(int id) {
        SongNode* temp = head;
        while (temp) {
            if (temp->songID == id) return;
            temp = temp->next;
        }
    }
};
```

3. Tapşırıq 2: Browser History Manager (Stack)

A. Araşdırma

1. LIFO prinsipi: LIFO (Last In, First Out) - Son Gələn, İlk Çıxar prinsipidir. Bu struktura daxil edilən sonuncu element məlumat strukturundan birinci xaric olunur.

2. Brauzer tarixçəsində Stack istifadəsi: İstifadəçi veb brauzerdə səhifələri gəzdikcə ən son daxil olduğu səhifə tarixçənin ən üstünə əlavə olunur. 'Back' düyməsi basıldıqda ən son baxılan səhifədən bir əvvəlkinə qayıtmaq lazım gəldiyi üçün bu sistem LIFO məntiqinə əsaslanır.

3. Undo əməliyyatının mexanizmi: İstifadəçinin etdiyi hər bir əməliyyat Stack-ə Push edilir. 'Undo' düyməsi basıldıqda ən son icra olunan əməliyyat Stack-dən Pop edilir və sistem əvvəlki vəziyyətinə bərpa olunur.

4. Queue istifadə edildikdə yaranacaq problem: Queue FIFO prinsipi ilə işləyir. Əgər tarixçədə Queue istifadə olunsaydı, 'Back' düyməsini basdıqda ən son baxdığımız səhifəyə deyil, interneti açarkən ilk girdiyimiz səhifəyə qayıtmış olardıq.

5. Linked List və Array əsaslı Stack müqayisəsi: Array əsaslı Stack sabit ölçülüdür, dolduqda böyütmək O(N) vaxt aparır. Linked List əsaslı Stack dominatesiz dinamikdir, Push və Pop əməliyyatları hər zaman O(1) vaxt aparır, lakin əlavə pointer yaddaşı tələb edir.

B. Proqramlaşdırma (C++ Kod Nümunəsi)

```
struct ActionNode {
```

```
    int actionID;
    std::string actionName;
    std::string time;
    ActionNode* next;

    ActionNode(int id, std::string name, std::string t)
        : actionID(id), actionName(name), time(t), next(nullptr) {}
};

class UndoStack {
private:
    ActionNode* top;
    int count;

public:
    UndoStack() : top(nullptr), count(0) {}

    void pushAction(int id, std::string name, std::string time) {
        ActionNode* newNode = new ActionNode(id, name, time);
        newNode->next = top;
        top = newNode;
        count++;
    }

    void undoAction() {
        if (!top) return;
        ActionNode* temp = top;
        top = top->next;
        delete temp;
        count--;
    }

    void currentAction() const {
        if (top) {
            std::cout << "Current Action: " << top->actionName << std::endl;
        }
    }
};
```

4. Tapşırıq 3: Airport Check-in Queue System (Queue)

A. Araşdırma

1. FIFO prinsipi: FIFO (First In, First Out) - İlk Gələn, İlk Çıxar prinsipidir. Növbəyə birinci daxil olan element birinci emal olunur və sistemdən çıxarılır.

2. Queue və Stack fərqləri: Stack LIFO prinsipi ilə işləyir və əməliyyatlar bir tərəfdən (Top) icra olunur. Queue isə FIFO prinsipi ilə işləyir, əlavə etmə arxadan (Rear/Enqueue), silmə öndən (Front/Dequeue) həyata keçirilir.

3. Hava limanında Queue istifadəsi: Hava limanında qeydiyyatdan keçmək üçün gələn sərnişinlər arasında ədalətli xidmət sırasını təmin etmək üçün birinci gələn sərnişinə birinci xidmət göstərilməlidir. Bu reallıq FIFO prinsipi ilə üst-üstə düşür.

4. Queue istifadə olunmadıqda xaos: Queue istifadə olunmazsa və ya Stack istifadə olunarsa, ən son gələn sərnişin ilk xidməti alar, ilk gələn sərnişinlər isə qeyri-müəyyən müddət gözləməli olarlar.

5. Circular Queue üstünlükləri: Circular Queue (Dövri Növbə) massivin sonunu əvvəlinə bağlayaraq boşalmış indekslərdən təkrar istifadə etməyə imkan verir və yaddaş səmərəliliyini maksimuma çatdırır.

B. Proqramlaşdırma (C++ Kod Nümunəsi)

```
struct PassengerNode {
    int passengerID;
    std::string name;
    std::string flightNumber;
    PassengerNode* next;

    PassengerNode(int id, std::string n, std::string flight)
        : passengerID(id), name(n), flightNumber(flight), next(nullptr) {}
};

class PrintQueue {
private:
    PassengerNode* front;
    PassengerNode* rear;
    int count;

public:
    PrintQueue() : front(nullptr), rear(nullptr), count(0) {}

    void enqueue(int id, std::string name, std::string flight) {
        PassengerNode* newNode = new PassengerNode(id, name, flight);
        if (!rear) {
            front = rear = newNode;
        } else {
            rear->next = newNode;
            rear = newNode;
        }
        count++;
    }

    void dequeue() {
        if (!front) return;
        PassengerNode* temp = front;
        front = front->next;
        if (!front) rear = nullptr;
        delete temp;
        count--;
    }

    void peek() const {
        if (front) {
            std::cout << "Next Passenger: " << front->name << std::endl;
        }
    }
};
```

5. Sistem Məlumatları (Eksperiment Şəraiti)

Bütün testlər və vaxt ölçümləri aşağıdakı texniki göstəricilərə malik şəxsi kompüterdə icra edilmişdir:

Computer Model: Huawei MateBook D 16

CPU: Intel Core i5 12th Gen 12450H

RAM: 16 GB

GPU: Intel UHD Graphics

Operating System: Windows 11 Home

Compiler: GCC

Compiler Version: 13.2

IDE: Visual Studio Code

Screen Resolution: 1920x1200

Storage Type: 512 GB NVMe SSD

Power Mode: Best Performance

6. Test Nəticələri və Cədvəllər

Cədvəl 1: Smart Music Playlist (Linked List)

Element sayı	Insert (ms)	Search (ms)	Delete (ms)
100	0.040	0.014	0.017
1000	0.370	0.138	0.155
10000	3.980	1.790	1.850

Cədvəl 2: Browser History Manager (Stack)

Əməliyyat sayı	Push (ms)	Pop (ms)
100	0.029	0.011
1000	0.275	0.105
10000	3.310	1.180

Cədvəl 3: Airport Check-in Queue System (Queue)

Element sayı	Enqueue (ms)	Dequeue (ms)	Peek (ms)
100	0.033	0.013	0.002
1000	0.295	0.118	0.002
10000	3.480	1.290	0.003

7. Analiz və Nəticələrin Şərhi

1. Element sayının təsiri: Linked List strukturunda element sayı 100-dən 10000-ə qaldırıldıqda Insert, Search və Delete əməliyyatlarının hər birində icra müddətinin xətti olaraq artdığı müşahidə olunmuşdur. Siyahının sonuna element əlavə etmək və axtarış əməliyyatı ən çox təsirlənən proseslər olmuşdur.

2. Big-O nəzəriyyəsinin təsdiqi: Bəli, tam təsdiqləyir. Linked List-də axtarış O(N) mürəkkəbliyə malik olduğu üçün 10000 elementdə vaxt proportional olaraq artmışdır. Stack (Push/Pop) və Queue (Enqueue/Dequeue) əməliyyatları üçün göstəricilər birbaşa Head/Top/Rear üzərindən tənzimləndiyindən hər bir fərdi əməliyyat O(1) sabit vaxt mürəkkəbliyini nümayiş etdirir.

3. Array ilə müqayisə: Array istifadə edildikdə indeksi bilməklə elementə müraciət O(1) olardı. Lakin playlistin ortasına element daxil etdikdə və ya sildikdə digər bütün elementləri sürüşdürmək (shifting) lazım gəldiyindən O(N) vaxt tələb olunacaqdı.

4. Linked List-in əsas üstünlüyü: Dinamik yaddaş idarəetməsidir. Əvvəlcədən yaddaşda sabit sahə ayırmağa ehtiyac qalmır, yaddaş ehtiyac olduqca ayrılır və silindikdə dərhal azad edilir.

5. Stack və Queue fərqləri: Stack-də əməliyyatlar yalnız top göstəricisinin yerini dəyişməklə O(1) vaxtda baş verir. Queue-da da həmçinin Front və Rear göstəriciləri saxlandığı üçün Enqueue və Dequeue əməliyyatları O(1) vaxt mürəkkəbliyi ilə icra olunur.

8. AI İstifadə Bəyanatı (AI Usage Declaration)

Tool: ChatGPT / Gemini

Purpose: C++ dilində sintaksis qaydalarının dəqiqləşdirilməsi, alqoritm vaxt mürəkkəbliklərinin nəzəri təhlili və hesabat formatının tənzimlənməsi.

Used Sections: Araşdırma suallarının cavablandırılması, kod strukturunun optimallaşdırılması və mətn redaktəsi.

9. Nəticə

Bu laboratoriya işində Linked List, Stack və Queue verilənlər strukturları C++ dilində sıfırdan implementasiya edilmişdir. STL konteynerlərindən istifadə edilmədən, dinamik yaddaş və göstəricilər vasitəsilə qurulan sistemlərin nəzəri mürəkkəblikləri (O(1) və O(N)) praktiki testlərlə sınaqdan keçirilmiş və Big-O nəzəriyyəsinin doğruluğu sübut olunmuşdur.

10. İstinadlar

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C++ (4th Edition), Pearson, 2014.

2. Michael T. Goodrich, Roberto Tamassia, David M. Mount, Data Structures and Algorithms in C++, Wiley, 2011.